

- Calcolando il valore $M[u]$ in pre-visita, ovvero quando u viene scoperto: cioè memorizzando il valore massimo trovato finora e settando $M[u]$ con il massimo fra u e il massimo valore incontrato finora. Questo ha il problema che non considera valori più alti che possono essere incontrati dopo, quindi l'algoritmo è sbagliato.
- Calcolando il valore $M[u]$ in post-visita, ovvero dopo aver visitato tutti i vicini e prendendo il massimo da tutti i vicini. Questo sembra un approccio migliore, ma ha comunque un problema: u prende il valore massimo fra tutti i discendenti, e questo è corretto; ma non è detto che i discendenti di u prendano il valore massimo di tutta la componente, perché quando vengono visitati non tutti i nodi sono stati scoperti. A titolo di esempio, considerate tre nodi come in figura



Supponendo che l'ordine di scoperta sia $2 - 1 - 3$, il nodo con valore 1 avrà valore massimo 2 e non 3.

Esercizio B1 – Numeri zerabili (0-based) – Punti ≥ 8

Il problema può essere risolto tramite la tecnica del backtrack, con un meccanismo di pruning che interrompe la ricorsione quando le stringhe contengono già un numero eccessivo di cifre 1.

Costruiamo la stringa binaria $S[0 \dots n-1]$ da sinistra a destra con backtracking. La chiamata $\text{zeroRec}(S, n, i, \text{ones})$ ha già fissato le posizioni $0 \dots i-1$ e ones è il numero di cifre 1 nel prefisso costruito. Proviamo sempre a porre $S[i] = 0$ (non aumenta il numero di 1). Proviamo invece $S[i] = 1$ solo se, dopo questa scelta, resta ancora possibile avere più 0 che 1 nell'intera stringa: ciò è garantito dalla condizione

$$\text{ones} + 1 < n - (\text{ones} + 1),$$

che equivale a richiedere che il numero totale di 1 rimanga strettamente minore del numero totale di 0. (Osservazione: la condizione può essere semplificata in $2 \cdot (\text{ones} + 1) < n$.)

Per induzione sulle chiamate ricorsive si mantiene l'invariante che il valore corrente di ones può ancora portare, completando le posizioni rimanenti, a una stringa con più 0 che 1. In particolare, ogni ramo che aggiunge un 1 rispetta la condizione sopra e quindi non può condurre a una stringa non zerabile; di conseguenza, quando $i = n$ la stringa stampata è zerabile. Viceversa, ogni stringa zerabile può essere seguita nelle scelte bit per bit senza mai violare la condizione, quindi viene generata e stampata.

```
printZerable(int n)
```

```
int[] S = new int[0...n-1]
zeroRec(S, n, 0, 0)
```

```
zeroRec(int [] S, int n, int i, int ones)
```

```
if i == n then
    print S[0...n-1]
else
    S[i] = 0
    zeroRec(S, n, i+1, ones)
    if ones + 1 < n - (ones + 1) then
        S[i] = 1
        zeroRec(S, n, i+1, ones + 1)
```

Un algoritmo che generasse tutte le stringhe binarie di lunghezza n e le stampasse avrebbe costo $\Theta(n2^n)$, poiché vengono prodotte 2^n stringhe e la stampa di ciascuna richiede tempo $\Theta(n)$.

Nel nostro caso, grazie al pruning, vengono generate e stampate solo le stringhe zerabili, il cui numero è

$$\sum_{k=0}^{\lfloor (n-1)/2 \rfloor} \binom{n}{k},$$

cioè tutte le stringhe con un numero di 1 strettamente minore di $n/2$. Questo valore è esponenziale in n , ma circa la metà di 2^n (più precisamente, è asintoticamente $\Theta(2^{n-1})$).

Infatti, il numero di stringhe zerabili di lunghezza $n = 1 \dots 10$ è

n	1	2	3	4	5	6	7	8	9	10
stringhe zerabili#	1	1	4	5	16	22	64	93	256	386

Questa è la sequenza <https://oeis.org/A058622> dell'Online Encyclopedia of Integer Sequences.

Esercizio B2 – Junior vs senior – Punti ≥ 10

Il problema può essere modellato come un problema di flusso massimo su un grafo bipartito, in cui i volontari vengono assegnati ai servizi. I volontari sono di due tipi, *junior* e *senior*, e per ogni servizio è imposto un vincolo minimo sul numero di volontari senior assegnati.

Costruzione della rete. Costruiamo una rete di flusso con i seguenti nodi:

- una sorgente s e un pozzo t ;
- un nodo vJ_i per ogni volontario junior;
- un nodo vs_i per ogni volontario senior;
- per ogni servizio s_j , due nodi sJ_j e sS_j , che rappresentano rispettivamente i volontari junior e senior assegnati al servizio s_j ;
- un nodo s_j per ogni servizio.

Gli archi della rete sono definiti come segue:

- un arco (s, vJ_i) e un arco (s, vs_i) con capacità 1, per ogni volontario: ogni volontario può essere assegnato ad al più un servizio;
- per ogni volontario vJ_i e servizio $s_j \in \text{avail}_i$, un arco (vJ_i, sJ_j) con capacità 1;
- per ogni volontario vs_i e servizio $s_j \in \text{avail}_i$, un arco (vs_i, sS_j) con capacità 1;
- per ogni servizio s_j , un arco (sJ_j, s_j) con capacità $vt_j - vs_j$, che limita il numero massimo di volontari junior assegnabili a vs_j ;
- per ogni servizio s_j , un arco (sS_j, s_j) con capacità vs_j , che consente l'assegnamento dei volontari senior;
- per ogni servizio s_j , un arco (s_j, t) con capacità vs_j .

Correttezza (intuizione). Ogni unità di flusso rappresenta l'assegnamento di un volontario a un servizio.

- La capacità 1 sugli archi in uscita dalla sorgente garantisce che ogni volontario sia assegnato ad al più un servizio.
- La separazione tra nodi sJ_j e sS_j consente di distinguere tra volontari junior e senior assegnati a ciascun servizio.
- L'arco (sJ_j, s_j) con capacità $vt_j - vs_j$ impone che al più $vt_j - vs_j$ volontari junior possano essere assegnati al servizio s_j ; di conseguenza, almeno vs_j volontari assegnati a s_j devono essere senior.
- L'arco (s_j, t) con capacità vs_j garantisce che il numero totale di volontari assegnati a s_j non superi vt_j .

Ne segue che esiste un'assegnazione valida se e solo se il flusso massimo satura tutti gli archi (s_j, t) fino a un valore complessivo pari a $\sum_j vt_j$.

Complessità. Il numero di nodi della rete è $|V| = 2 + n + 3m = O(n + m)$ (sorgente e pozzo, n nodi volontario, e per ogni servizio i nodi sJ_j , sS_j e s_j).

Gli archi sono:

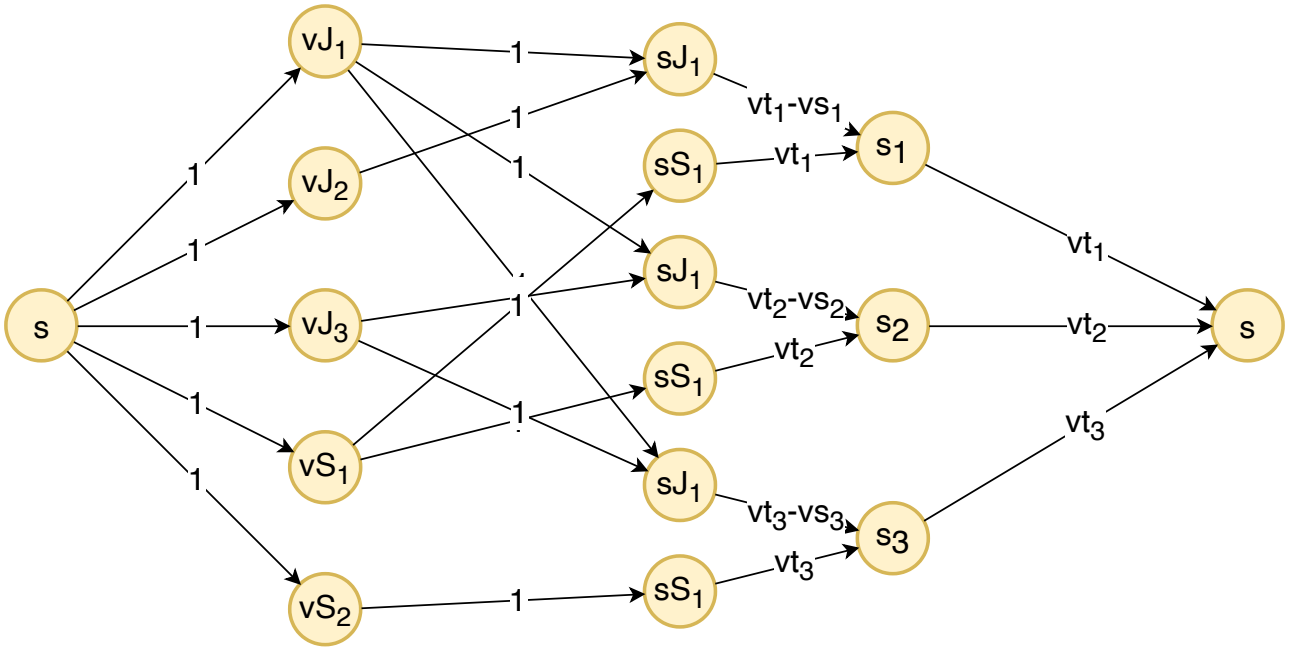
- n archi dalla sorgente ai volontari;
- un arco da ciascun volontario ai servizi per cui è disponibile: in totale $O(nm)$ archi;
- $2m$ archi dai nodi sJ_j, sS_j ai nodi s_j ;
- m archi dai nodi s_j al pozzo.

Quindi $|E| = n + O(nm) + 3m = O(nm)$.

Il valore del flusso massimo è al più n (ogni volontario contribuisce al più 1). Usando Ford–Fulkerson, si ottiene

$$O(|f^*|(|V| + |E|)) = O(n(nm)) = O(n^2m).$$

Un esempio di rete di flusso per 5 volontari, 3 servizi e un vincolo minimo sui volontari senior è mostrato in Figura ??.



Esercizio B3 – Borsa (0-based) – Punti ≥ 12

Il problema può essere risolto tramite programmazione dinamica.

Sia $DP[i][\ell]$ il massimo guadagno che si può ottenere considerando i primi i giorni (vettore $prices[0 \dots i-1]$) e dovendo fare esattamente ℓ operazioni. $DP[i][\ell]$ può essere calcolato ricorsivamente in questo modo:

$$DP[i][\ell] = \begin{cases} 0 & \ell = 0 \\ -\infty & \ell > 0 \text{ and } i < 2\ell \\ \max\{DP[i-1][\ell], \max_{1 \leq j \leq i-1} \{DP[j-1][\ell-1] + (prices[i-1] - prices[j-1])\}\} & \text{otherwise} \end{cases}$$

La spiegazione è la seguente:

- se ho zero operazioni da fare, il massimo guadagno è zero, perché non posso acquistare e vendere, qualunque sia il numero di giorni;
- se il numero di giorni residui non è sufficiente per effettuare ℓ operazioni, il guadagno è $-\infty$ perché non è possibile rispettare la regola del numero esatto di operazioni;
- altrimenti, ci sono due possibilità:
 - decido di ignorare il giorno in posizione $i-1$, restando con $i-1$ giorni residui e ℓ operazioni ancora da fare;
 - decido di vendere nel giorno i -esimo (posizione nel vettore $i-1$), cercando il massimo fra tutti i giorni di acquisto j -esimi, con j compreso fra 1 (primo giorno) e $i-1$ (precedente al i -esimo giorno di vendita) e restando con $j-1$ giorni rimanenti, $\ell-1$ operazioni.

Il valore desiderato $DP[n][k]$ può essere calcolato iterativamente nel modo seguente:

maxGain(int[] prices, int n, int k)

```
int[][] DP = newint[0...n][0...k]
for i = 0 to n do
  DP[i][0] = 0
for ℓ = 1 to k do
  DP[0][ℓ] = -∞
for i = 1 to n do
  for ℓ = 1 to k do
    if i < 2ℓ then
      DP[i][ℓ] = -∞
    else
      DP[i][ℓ] = DP[i-1][ℓ]
      for j = 1 to i-1 do
        DP[i][ℓ] = max(DP[i][ℓ], DP[j-1][ℓ-1] + (prices[i-1] - prices[j-1]))
return DP[n][k]
```

La complessità dell'algoritmo è pari a $O(kn^2)$, derivante dal fatto che ogni casella costa $O(n)$ per essere riempita e ci sono $\Theta(nk)$ caselle.